¿Con qué rapidez cierran los modelos ejecutables localmente la brecha de programación? Una revisión sistemática PRISMA 2020, 2023-2026

Juan Sebastian Perez Galvis

1. Introducción

Los modelos de lenguaje de gran tamaño (LLM) cambiaron la forma de escribir y mantener código, y buena parte de esa capacidad ya no depende de la nube. Desde 2023 se distribuyen pesos abiertos que corren en un computador personal, sin costo por consulta y sin enviar el código a terceros. Code Llama (Rozière et al., 2023), Mistral 7B (Jiang et al., 2023), CodeGemma (CodeGemma Team, 2024), StarCoder2 (Lozhkov et al., 2024), Llama 3 (Grattafiori et al., 2024), Phi-4 (Abdin et al., 2024), Gemma 3 (Gemma Team, 2025), DeepSeek-R1 (DeepSeek-AI, 2025), Qwen2.5-Coder (Hui et al., 2024) y Qwen3-Coder (Qwen Team, 2025) pertenecen a esa línea, y varias de sus variantes cuantizadas caben en una GPU de consumo. La pregunta es práctica. Quien elige hardware, la organización que no puede externalizar su código y quien no puede pagar suscripciones necesitan saber qué se ejecuta de verdad en una máquina personal y con qué rapidez mejora esa capacidad.

La capacidad de programación se mide con un repertorio de benchmarks ya asentado. HumanEval (Chen et al., 2021) y MBPP (Austin et al., 2021) comprueban la corrección funcional de funciones aisladas. HumanEval+ y MBPP+ (Liu et al., 2023) endurecen esas pruebas, y LiveCodeBench (Jain et al., 2024) y BigCodeBench (Zhuo et al., 2024) usan problemas recientes y resistentes a la contaminación. SWE-bench Verified (Jimenez et al., 2024; OpenAI, 2024) lleva la medición a la resolución de incidencias reales de repositorios. Sobre esa base, los informes técnicos y las tablas de posición públicas, como Epoch AI Benchmarking Hub (Epoch AI, 2026), EvalPlus, LiveBench (White et al., 2024) y SWE-bench Verified, registran una mejora sostenida de los modelos abiertos y una brecha que se estrecha frente a la frontera propietaria. La literatura también ha estudiado la diferencia general entre modelos abiertos y cerrados, que abarca desde variantes de consumo hasta modelos de escala de centro de datos (Zhu et al., 2024). Junto a ello, se ha medido el efecto de la cuantización sobre la calidad del código (Dettmers y Zettlemoyer, 2023; Frantar et al., 2023) y las condiciones del despliegue en dispositivos (Xu et al., 2024).

Esa evidencia no responde, sin embargo, a la pregunta de quien compra hardware de consumo. Las tablas de posición mezclan modelos que exigen centros de datos con modelos cuantizados que caben en un portátil, y no incluyen una columna de viabilidad. Sus cifras describen el máximo alcanzable con recursos irrestrictos, no lo que ocurre en una máquina personal. Los estudios que sí evalúan modelos locales se concentran en benchmarks cercanos a la saturación (Jain et al., 2024), y la capacidad de ingeniería de software medida con SWE-bench apenas aparece reportada para estos modelos. La frontera se mide sin restricción de viabilidad y lo local se mide sin benchmarks exigentes. Hasta donde alcanza nuestra búsqueda, nadie ha medido de forma sistemática y reproducible la velocidad con que

los modelos ejecutables localmente cierran la brecha frente a la frontera propietaria, ni cuánto de esa brecha proviene de la restricción de hardware. Ese vacío es el que aborda esta revisión.

El objetivo general es cuantificar la evolución del desempeño en tareas de programación de los modelos ejecutables localmente y medir la brecha que los separa de la frontera propietaria entre 2023 y 2026. La revisión responde dos preguntas: *¿con qué rapidez, entre 2023 y 2026, reducen la brecha los modelos ejecutables localmente frente a los modelos cerrados de la frontera propietaria?* y *¿qué tan grande es la brecha restante en cada benchmark y fecha?* El marco PICOS precisa el alcance de ambas. La población son los modelos que caben en hardware de consumo, con pesos `Q4_K_M` y caché de contexto residente: 16 GB de memoria de video como escenario principal, 24 GB como cota superior y 8 GB como escenario de sensibilidad. La exposición es la fecha de publicación del modelo, cuyo coeficiente se estima. El comparador combina la frontera propietaria cerrada con una línea abierta sin filtro de viabilidad. Los desenlaces son los benchmarks preespecificados HumanEval(+), MBPP(+), LiveCodeBench y BigCodeBench, junto con SWE-bench Verified y Pro. Cierra el marco el diseño: una revisión sistemática con síntesis cuantitativa.

La revisión sigue las directrices PRISMA 2020 (Page et al., 2021) y PRISMA-S (Rethlefsen et al., 2021), con protocolo registrado en OSF y cribado doble. Cubre literatura en inglés publicada entre 2023 y 2026, con las evaluaciones de SWE-bench restringidas a 2024 y 2026. Quedan fuera los modelos cerrados como objeto de evaluación, los que solo operan en centros de datos y la reejecución propia de los benchmarks. La revisión descompone la brecha en dos partes: la diferencia abierto-cerrado, ya estudiada, y un costo de viabilidad que no se ha medido. También deja un corpus auditable para replicar el análisis. El artículo se organiza en cinco secciones. La sección 2 presenta los métodos, la sección 3 los resultados, la sección 4 la discusión y la sección 5 las conclusiones.

2. Referencias

Abdin, M., Jacobs, S. A., Awan, A. A., Aneja, J., Awadallah, A., Awadalla, H., … & Zhou, X. (2024). *Phi-4 technical report* (arXiv:2412.08905). arXiv. https://doi.org/10.48550/arXiv.2412.08905

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., … & Sutton, C. (2021). *Program synthesis with large language models* (arXiv:2108.07732). arXiv. https://doi.org/10.48550/arXiv.2108.07732

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., … & Zaremba, W. (2021). *Evaluating large language models trained on code* (arXiv:2107.03374). arXiv. https://doi.org/10.48550/arXiv.2107.03374

CodeGemma Team. (2024). *CodeGemma: Open code models based on Gemma* (arXiv:2406.11409). arXiv. https://doi.org/10.48550/arXiv.2406.11409

DeepSeek-AI. (2025). *DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning* (arXiv:2501.12948). arXiv. https://doi.org/10.48550/arXiv.2501.12948

Dettmers, T., & Zettlemoyer, L. (2023). The case for 4-bit precision: k-bit inference scaling laws. En *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)* (pp. 7750-7774). PMLR.

Epoch AI. (2026). *Benchmarking Hub* [Conjunto de datos]. https://epoch.ai/data

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. En *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*.

Gemma Team. (2025). *Gemma 3 technical report* (arXiv:2503.19786). arXiv. https://doi.org/10.48550/arXiv.2503.19786

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., … & Zhang, Z. (2024). *The Llama 3 herd of models* (arXiv:2407.21783). arXiv. https://doi.org/10.48550/arXiv.2407.21783

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., … & Lin, J. (2024). *Qwen2.5-Coder technical report* (arXiv:2409.12186). arXiv. https://doi.org/10.48550/arXiv.2409.12186

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., … & Stoica, I. (2024). *LiveCodeBench: Holistic and contamination free evaluation of large language models for code* (arXiv:2403.07974). arXiv. https://doi.org/10.48550/arXiv.2403.07974

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., … & Sayed, W. E. (2023). *Mistral 7B* (arXiv:2310.06825). arXiv. https://doi.org/10.48550/arXiv.2310.06825

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? En *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*.

Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. En *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Lozhkov, A., Li, R., Allal, L. B., Cassano, F., Lamy-Poirier, J., Tazi, N., … & de Vries, H. (2024). *StarCoder 2 and The Stack v2: The next generation* (arXiv:2402.19173). arXiv. https://doi.org/10.48550/arXiv.2402.19173

OpenAI. (2024, 19 de agosto). *Introducing SWE-bench Verified*. https://openai.com/index/introducing-swe-bench-verified/

Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., … & Moher, D. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, *372*, n71. https://doi.org/10.1136/bmj.n71

Qwen Team. (2025). *Qwen3-Coder: Agentic coding in the world*. Alibaba Group. https://qwenlm.github.io/blog/qwen3-coder/

Rethlefsen, M. L., Kirtley, S., Waffenschmidt, S., Ayala, A. P., Moher, D., Page, M. J., & Koffel, J. B. (2021). PRISMA-S: An extension to the PRISMA statement for reporting literature searches in

systematic reviews. *Systematic Reviews*, *10*(1), 39. https://doi.org/10.1186/s13643-020-01542-z

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., … & Synnaeve, G. (2023). *Code Llama: Open foundation models for code* (arXiv:2308.12950). arXiv. https://doi.org/10.48550/arXiv.2308.12950

White, C., Dooley, S., Roberts, M., Pal, A., Feuer, B., Jain, S., … & Goldblum, M. (2024). *LiveBench: A challenging, contamination-free LLM benchmark* (arXiv:2406.19314). arXiv. https://doi.org/10.48550/arXiv.2406.19314

Xu, J., Li, Z., Chen, W., Wang, Q., Gao, X., Cai, Q., & Ling, Z. (2024). *On-device language models: A comprehensive review* (arXiv:2409.00088). arXiv. https://doi.org/10.48550/arXiv.2409.00088

Zhu, Q., Guo, D., Shao, Z., Yang, D., Wang, P., Xu, R., … & Guo, D. (2024). *DeepSeek-Coder-V2: Breaking the barrier of closed-source code models* (arXiv:2406.11931). arXiv. https://doi.org/10.48550/arXiv.2406.11931

Zhuo, T. Y., Vu, M. C., Chim, J., Hu, H., Yu, W., Widyasari, R., … & Xia, C. S. (2024). *BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions* (arXiv:2406.15877). arXiv. https://doi.org/10.48550/arXiv.2406.15877